Tutorial: Implement the missing `Snake` game functions (Intermediate)

You are given the starter `Snake.cs` file (title + HSV already done, empty methods + fields provided). Implement the missing members to satisfy `IGame`.

Repo-specific note: claim code (6-digit) is injected

In this repo, the 6-digit claim code is minted by `ConsoleTest/Program.cs` and injected into the game via `SetGameOverCode(string? code)`. Your game should:

- Set `gameOver = true` when it ends
- Expose `IsGameOver()`
- Store and return the injected claim code via `SetGameOverCode` / `GetGameOverCode`

Do **not** call `CodeGenerator` inside the game.

Step 0: Fields you likely need

```
private readonly Queue<(int x, int y)> snake = new Queue<(int x, int y)>();
private int snakeLength = 5;
private int headX = 10;
private int headY = 5;

private int directionX = 0;
private int directionY = 1;
private int nextDirectionX = 0;
private int nextDirectionY = 1;

private readonly Random rand = new Random();
private (int x, int y) food;

private int score = 0;
private float rainbowShift = 0f;

private bool gameOver = false;
private bool wallsAreDeadly = false;
private int moveCounter = 0;

private string? gameOverCode;

// IGame requirement:
public string GameId { get; } = "REPLACE_WITH_REAL_GAME_ID";
```

Step 1: Implement `DrawTitle(IPixel[,] pixels)` (intent)

- Fill the board with something visually distinct from gameplay.
- Optional: animate using a slowly changing variable like `rainbowShift`.
- Optional: draw a simple logo/letter by setting some pixels to black.

Step 2: Implement `Initialize(IPixel[,] pixels)` (pseudocode)

- Clear snake queue
- Reset snake length, head position, direction, score, counters, and `gameOver`
- Reset `gameOverCode = null` (new run)
- Spawn food within bounds
- Create initial snake body segments behind the head

Step 3: Implement `Update(IPixel[,] pixels)` (pseudocode)

- If `gameOver`:
  - call `DrawGameOver(pixels)`
  - return
- `moveCounter++`
- If `moveCounter < GetMoveSpeed()`:
  - `DrawGame(pixels)`
  - return
- `moveCounter = 0`
- Apply buffered direction:
  - `directionX = nextDirectionX`
  - `directionY = nextDirectionY`
- Move head:
  - `headX += directionX`
  - `headY += directionY`
- Walls:
  - deadly: out of bounds => `gameOver = true`, `DrawGameOver`, return
  - wrap: clamp into [0..19] / [0..9] with wrap logic
- Self collision:
  - if `snake.Contains((headX, headY))` => `gameOver = true`, draw, return
- Food collision:
  - if head == food => `score++`, `snakeLength++`, respawn food in an empty cell

- Body update:
  - `snake.Enqueue((headX, headY))`
  - if `snake.Count > snakeLength` => `snake.Dequeue()`
- Draw:
  - `DrawGame(pixels)`

Important: do **not** generate the 6-digit claim code here.

Step 4: Implement `HandleInput(ConsoleKey key, ref bool stateChanged)` (pseudocode)

- If `gameOver`:
  - if Escape => `stateChanged = true`
  - return
- Movement keys:
  - update `nextDirectionX/nextDirectionY`
  - prevent reversing into yourself (block opposite direction)
- Escape:
  - `stateChanged = true`

Step 5: Implement `GetScore`, `IsGameOver`, `GetGameOverCode`, `SetGameOverCode`

- `GetScore()` returns `score`
- `IsGameOver()` returns `gameOver`
- `GetGameOverCode()` returns `gameOverCode` (nullable)
- `SetGameOverCode(code)` stores the injected value

Step 6: Implement `GetMoveSpeed()` (pseudocode)

- Return a smaller number as score increases
- Example:
  - score < 5 => 3
  - score < 10 => 2
  - else => 1

Step 7: Implement `DrawGame(IPixel[,] pixels)` (pseudocode)

- Fill background with dark color
- Draw snake body green, head brighter green

- Draw food red

Step 8: Implement `DrawGameOver(IPixel[,] pixels)` (pseudocode)

- Draw final frame
- Overlay a strong red tint to indicate game over
- Keep rendering the same game-over visuals when `Update` is called post-death